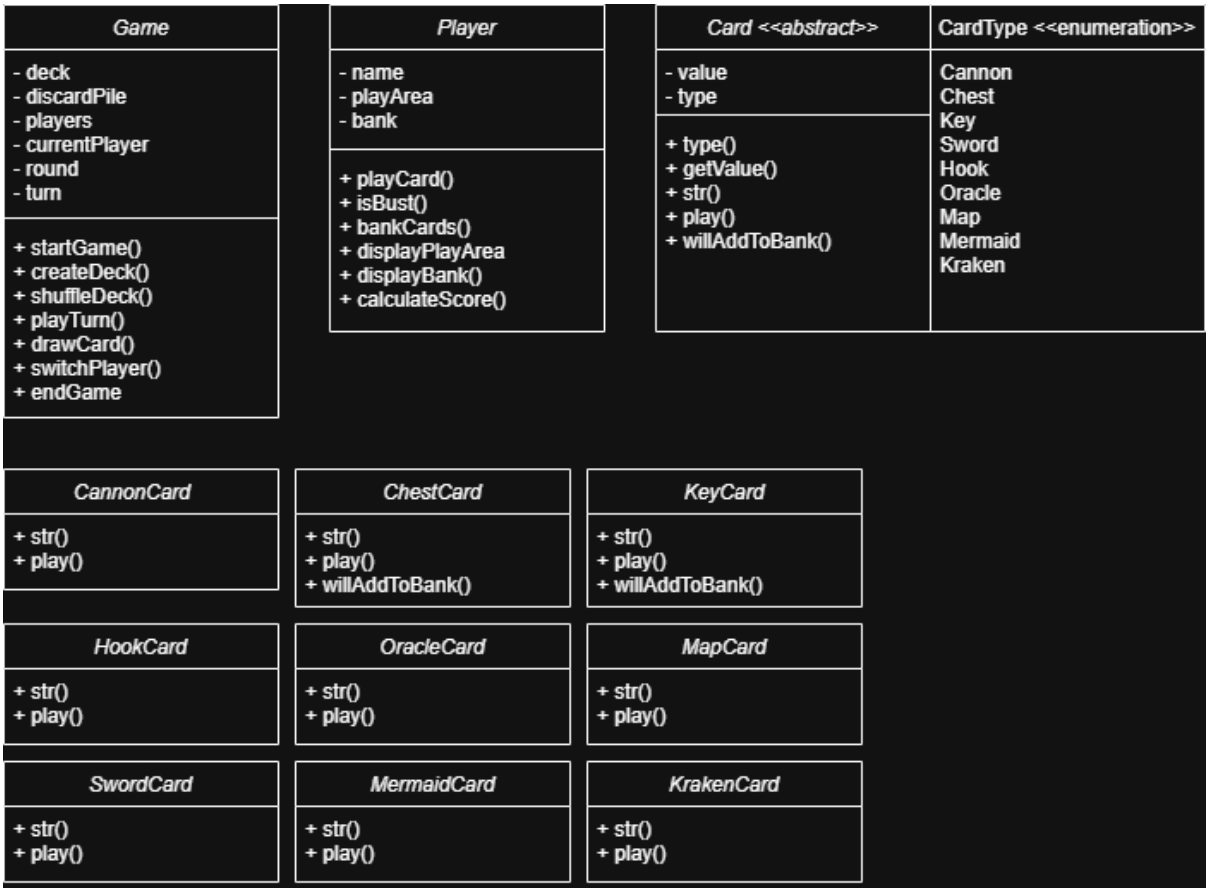
Design with C++ Assignment 1 Document

By Duy Nguyen

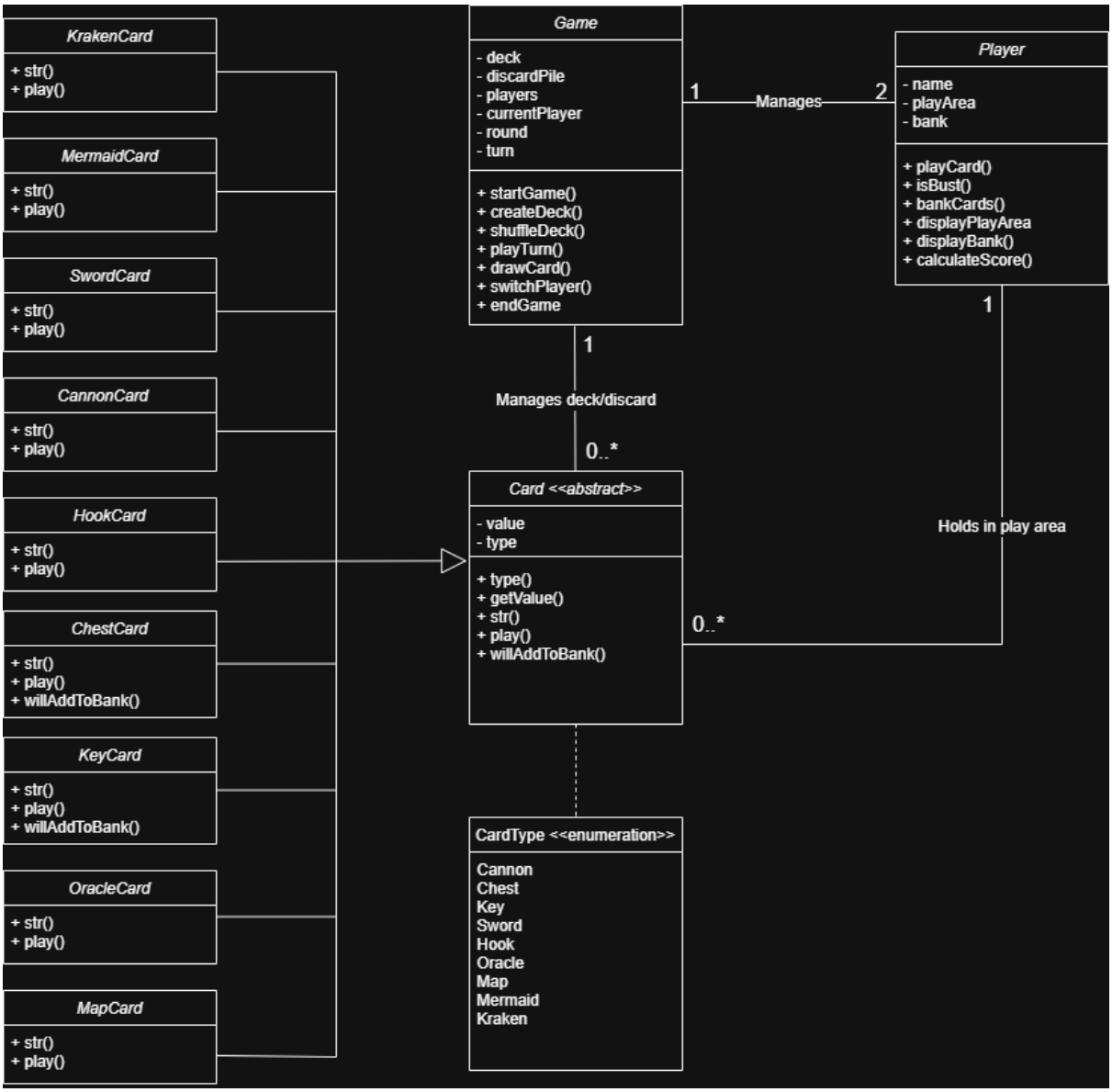
StudentID: A1887926

Email: A1887926@student.adelaide.edu.au

Classes with respective attributes and methods



UML Class Diagram



Design Decisions

- Game class is set as the main controller as it's responsible for starting the game, creating, shuffling, tracking turn and rounds, and switching between players.
- Card is set as an abstract class as all cards share common features, such as a value, type, and ability to be printed. However, each card suit has its own distinct behaviour. Setting card as an abstract class avoids unnecessary repetitions in every subclass.
- Player has both a play area and a bank, as the game rules treat them differently.
- The play card method was also introduced that adds a cards to the players play area and removes it from the deck, as suggested by the Assignment structure.
- The decision to keep score calculation inside player rather than game was decided due to players being the entities that own the bank. The game announces the winner, but each player is responsible for calculating their score from their banked cards.
- The method willAddToBank() in Chest and Key is included given their special behaviours. Their effects happens when cards are banked instead of immediately when drawn.
- The decision to not overcomplicate the design was intentional, not creating extra classes as Deck, Bank or PlayArea, as these classes could rather be handled inside classes such as Game and Player. The assignment already stated the main classes, the decision to add extra classes would make the design overly complicated.

- A one-to-two multiplicity was decided between Game and Player as this game's Dead Man's Draw++ was designed for two players.
- A one-to-many multiplicity between Game and Card was decided as the game controls/manages the deck, consisting of many cards.
- A one-to-many multiplicity between Player and Cards was decided as the player can also hold multiple cards in their play area and bank.
- Generalisation was used between card and the individual suit subclasses as all cards share the same basic structure. All subclasses were grouped under one generalisation branch to show they all inherit from the same abstract Card parent. This was done to keep the UML diagram cleaner and make the inheritance structure easier to read.